Optimization in Machine Learning

Avi Mohan

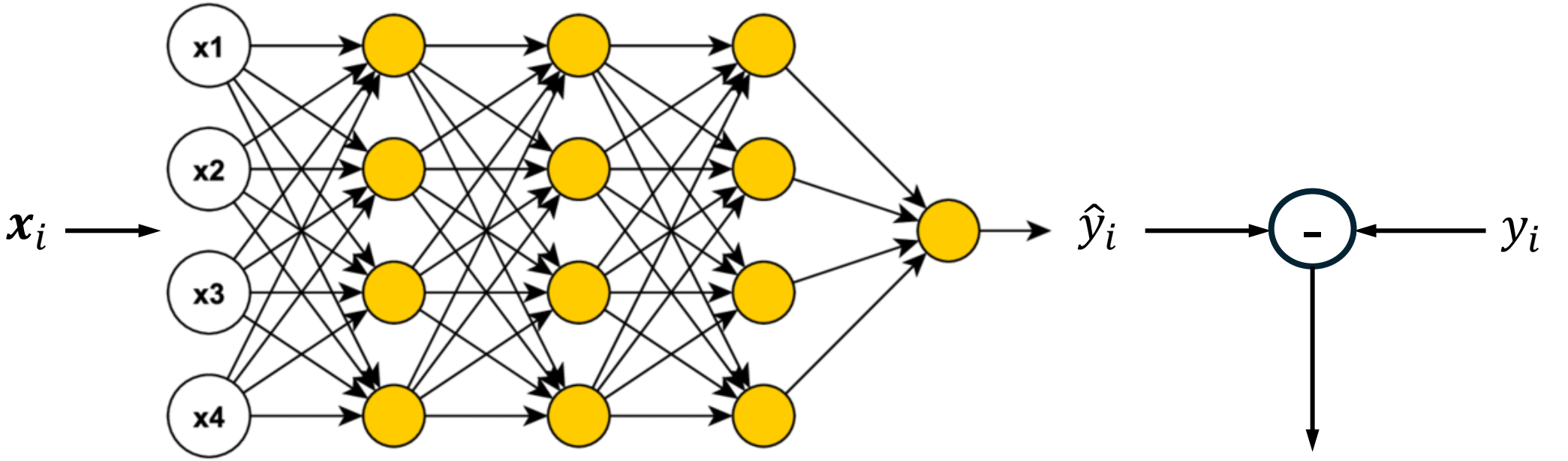
Today's Class

- Differentiation and optimizing functions of 1 variable
- Iterative minimization in 1D

- Understanding the Gradient
- Gradient descent
- Issues with gradient descent

- Faster algorithms

Situating the problem



$$\frac{1}{N_{train}} \sum_{i=1}^{N_{train}} (y_i - \hat{y}_i)^2$$

What is being optimized?

- Problem: find optimal weights (of the neural network) that minimize training loss.
- Recall: "*end-to-end*" view of NNs
  - output of NN is a function of the inputs and weights.
- Predicted label $\hat{y} = g(\boldsymbol{x}, \boldsymbol{w})$
- Training loss

$$\mathcal{L}(\boldsymbol{w}) = \frac{1}{N_{train}} \sum_{i=1}^{N_{train}} (y_i - g(\boldsymbol{x}_i, \boldsymbol{w}))^2$$

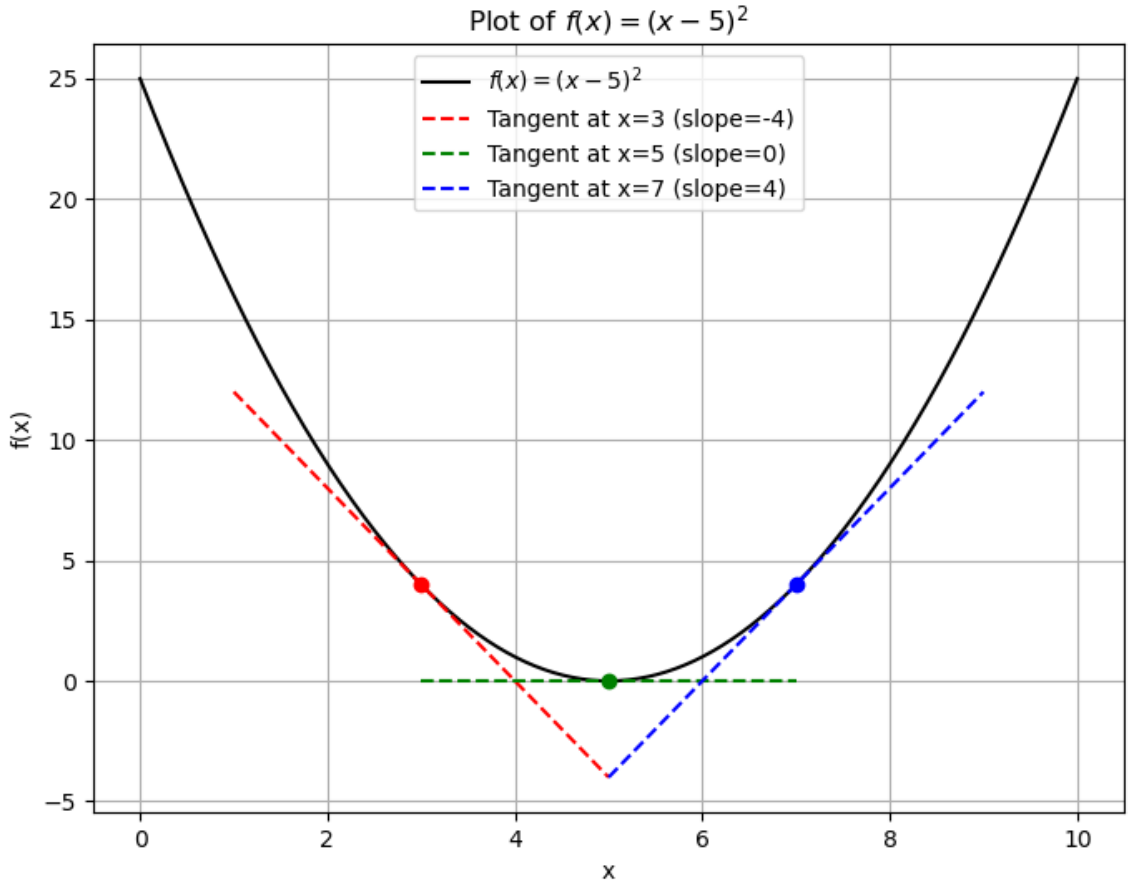
- Problem: find optimal weights that minimize $\mathcal{L}(\boldsymbol{w})$.

Optimization in 1D

- Let's begin with single-variable functions.
  - $f(x) = (x-5)^2$

- NASC: x^* minimizes f
  - $f'(x^*) = 0$, and
  - 2nd derivative (f'') is positive

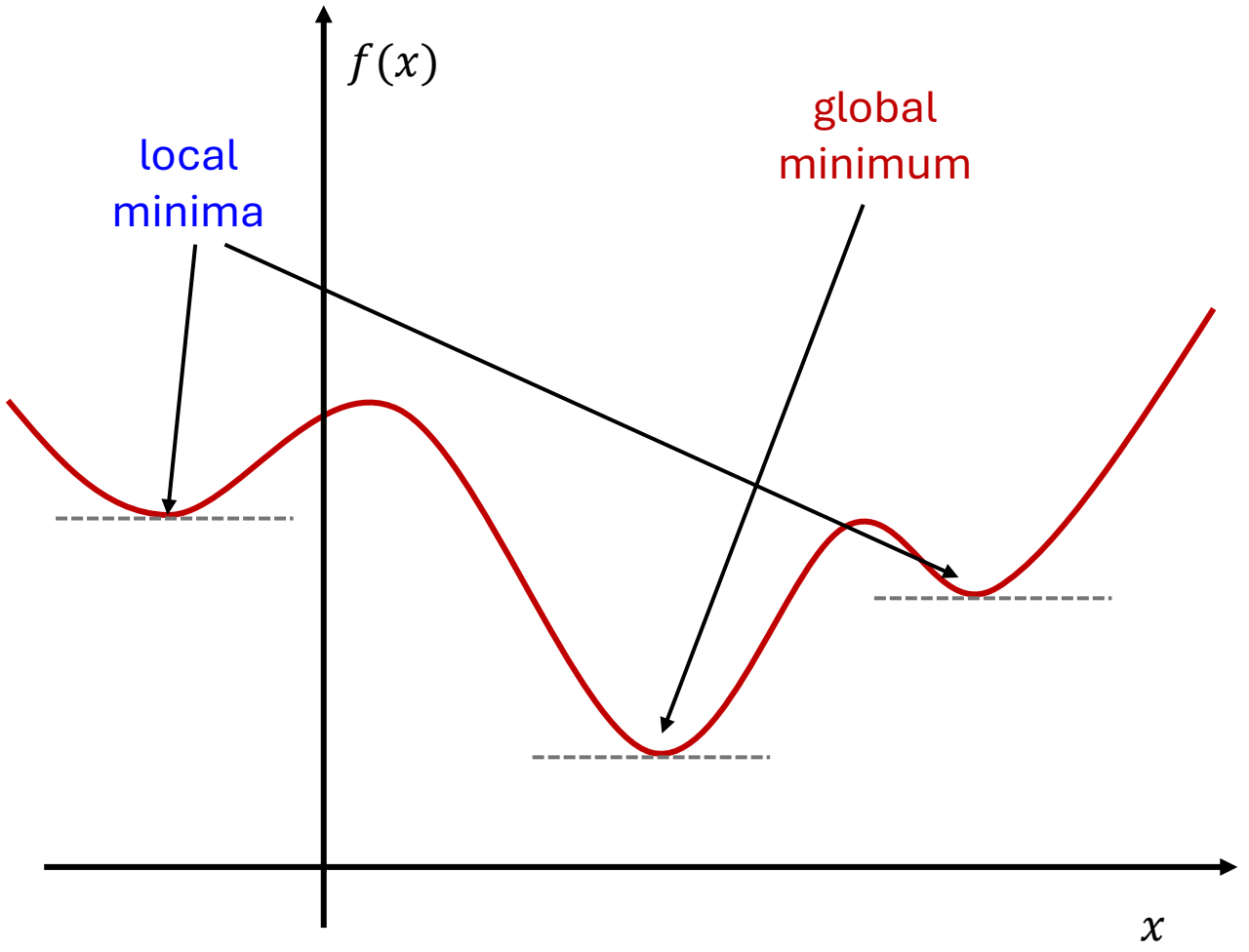
- We will only examine the first order condition.
- Unconstrained optimization only



Optimization in 1D

- NASC: x^* minimizes f
  - $f'(x^*) = 0$, and
  - 2nd derivative (f'') is positive

- ⚠️ these conditions only guarantee a LOCAL minimum.

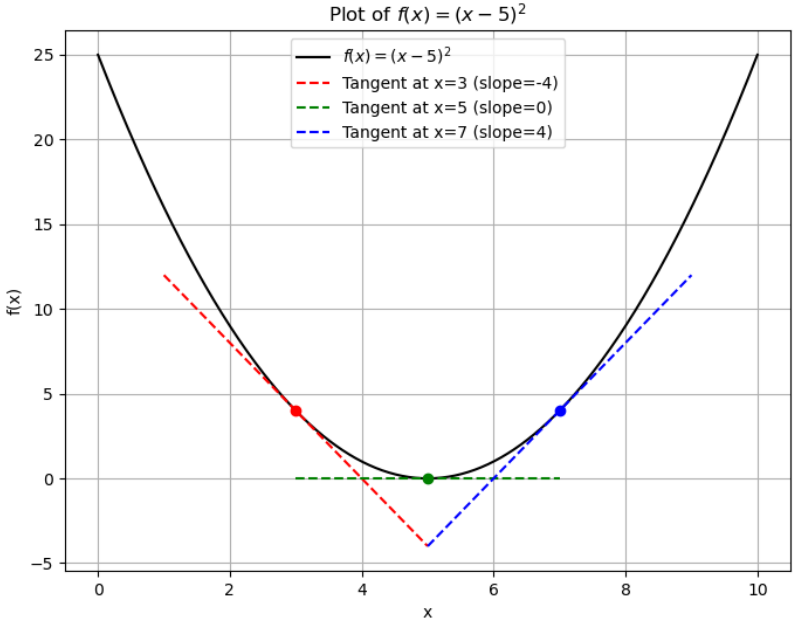


Optimization in 1D

- Back to our function
  - $f(x) = (x-5)^2$

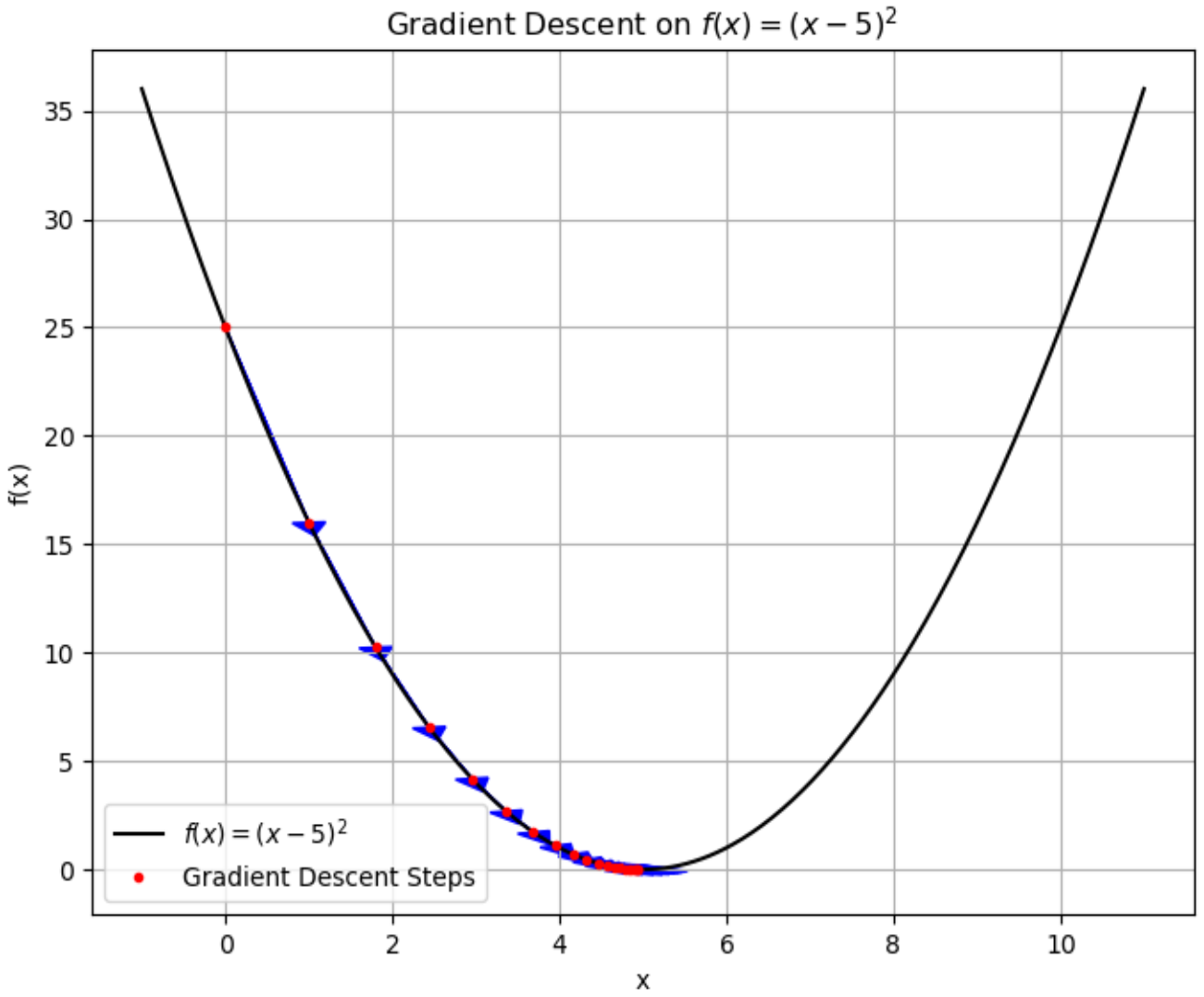
- Optimization algorithms: iteratively find the optimizer of a given function (x^*)
  - In each iteration tweak x to move closer to an optimal value.

- Gradient descent
  - First order, iterative algorithm
  - In each step t, tweak x_t proportional to $-f'(x_t)$



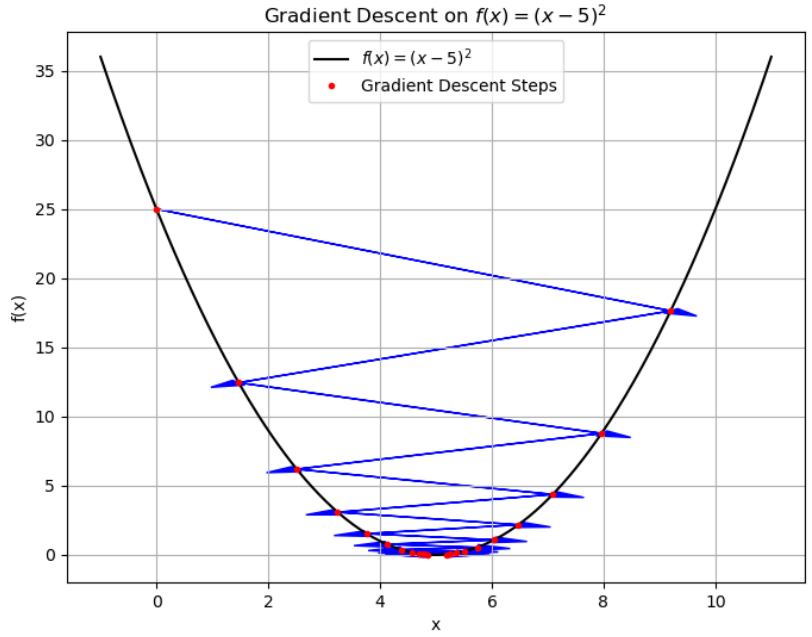
Gradient Descent

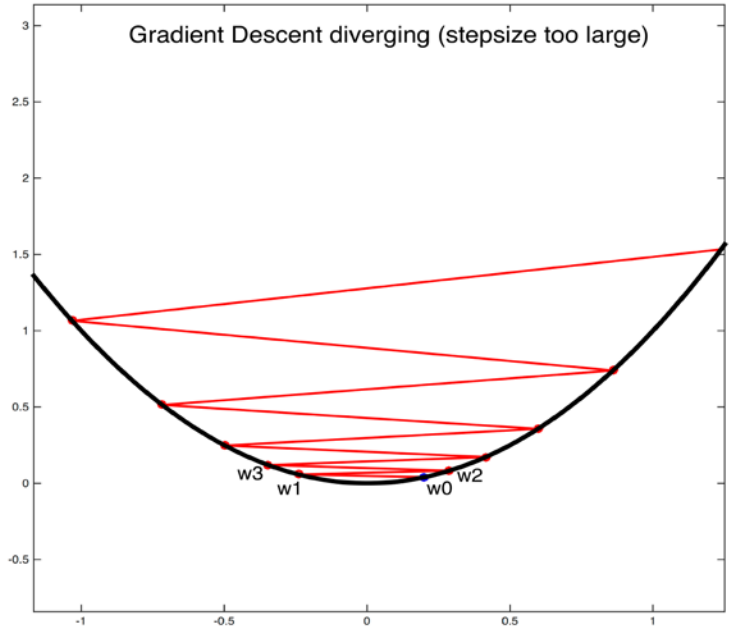
- Repeat until convergence:
  - $x_{t+1} = x_t - \eta f'(x_t)$
- Where,
  - x_0: initial condition
  - η: learning rate or step size
  - $(\eta = 0.1)$



Gradient Descent

- Repeat until convergence:
  - $x_{t+1} = x_t - \eta f'(x_t)$
- Where,
  - x_0: initial condition
  - η: learning rate or step size
- Very large learning rates can lead to unstable behavior!





Today's Class

- Differentiation and optimizing functions of 1 variable
- Iterative minimization in 1D

- Understanding the Gradient
- Gradient descent
- Issues with gradient descent

- Faster algorithms

Functions of several variables

- In machine (deep) learning, we work with models that have millions of parameters.
- Gradient (informally) : the generalization of the derivative to functions of several variables

- $f(x, y)$: function of both x and y
  - PARTIAL derivatives ($d \rightarrow \partial$)
  - $\frac{\partial}{\partial x} f$ and $\frac{\partial}{\partial y} f$

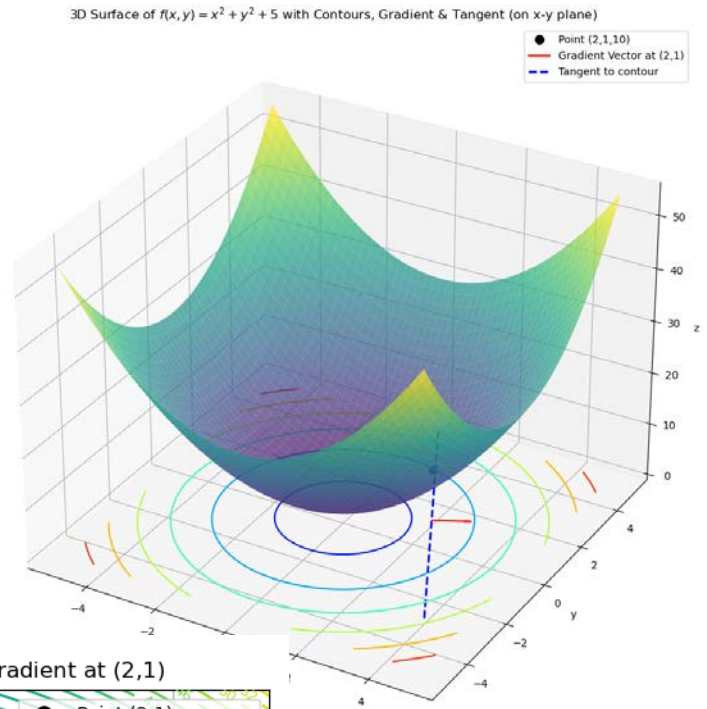
- $\nabla f = \left[\frac{\partial}{\partial x} f, \frac{\partial}{\partial y} f\right]$

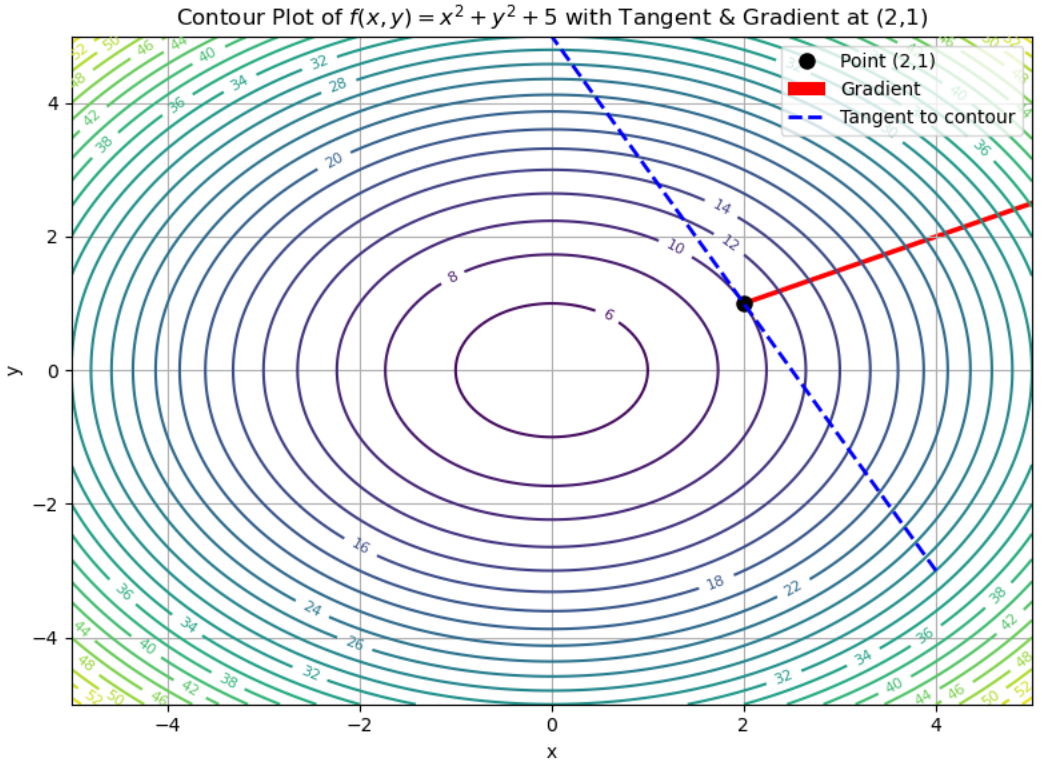
- $f(x, y)$: function of both x and y
  - PARTIAL derivatives ($d \rightarrow \partial$)
  - $\frac{\partial}{\partial x} f$ and $\frac{\partial}{\partial y} f$

- $\nabla f = \left[\frac{\partial}{\partial x} f, \frac{\partial}{\partial y} f\right]$

- What are “contour plots?”

- Defines the direction of steepest ASCENT.





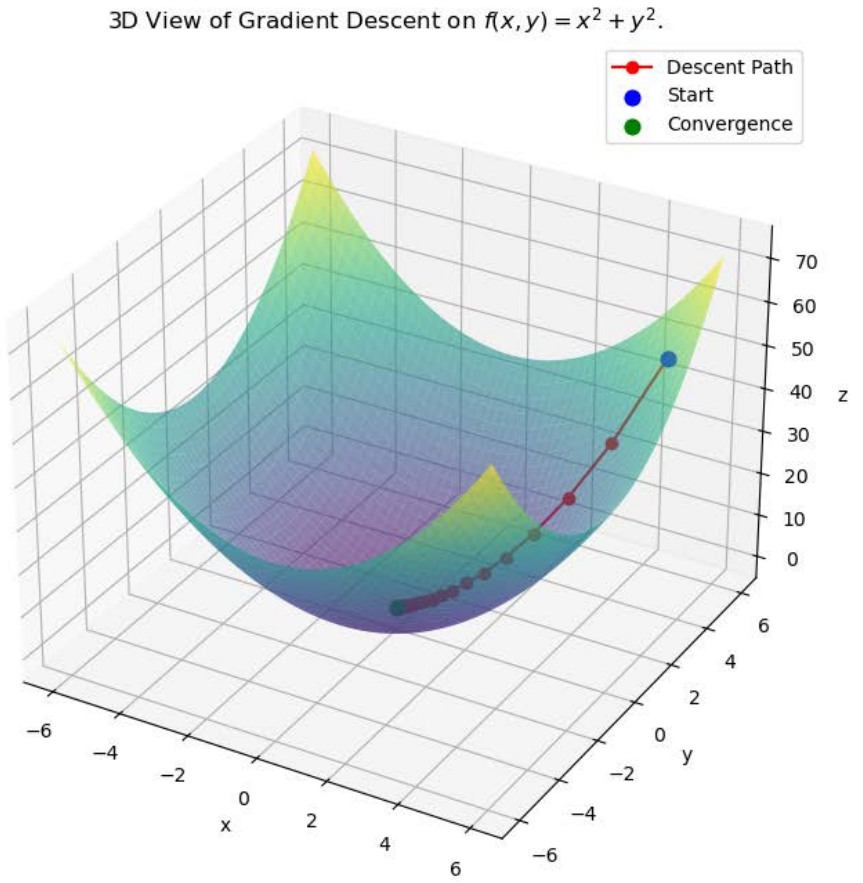
Gradient Descent

- $f(\boldsymbol{x}) = x_1^2 + x_2^2$
  - $\boldsymbol{x} = [x_1, x_2]$

- Repeat until convergence:
  - $\boldsymbol{x}_{t+1} = \boldsymbol{x}_t - \eta \nabla f(\boldsymbol{x}_t)$

- Where,
  - $\boldsymbol{x}_0$: initial condition
  - η: learning rate or step size

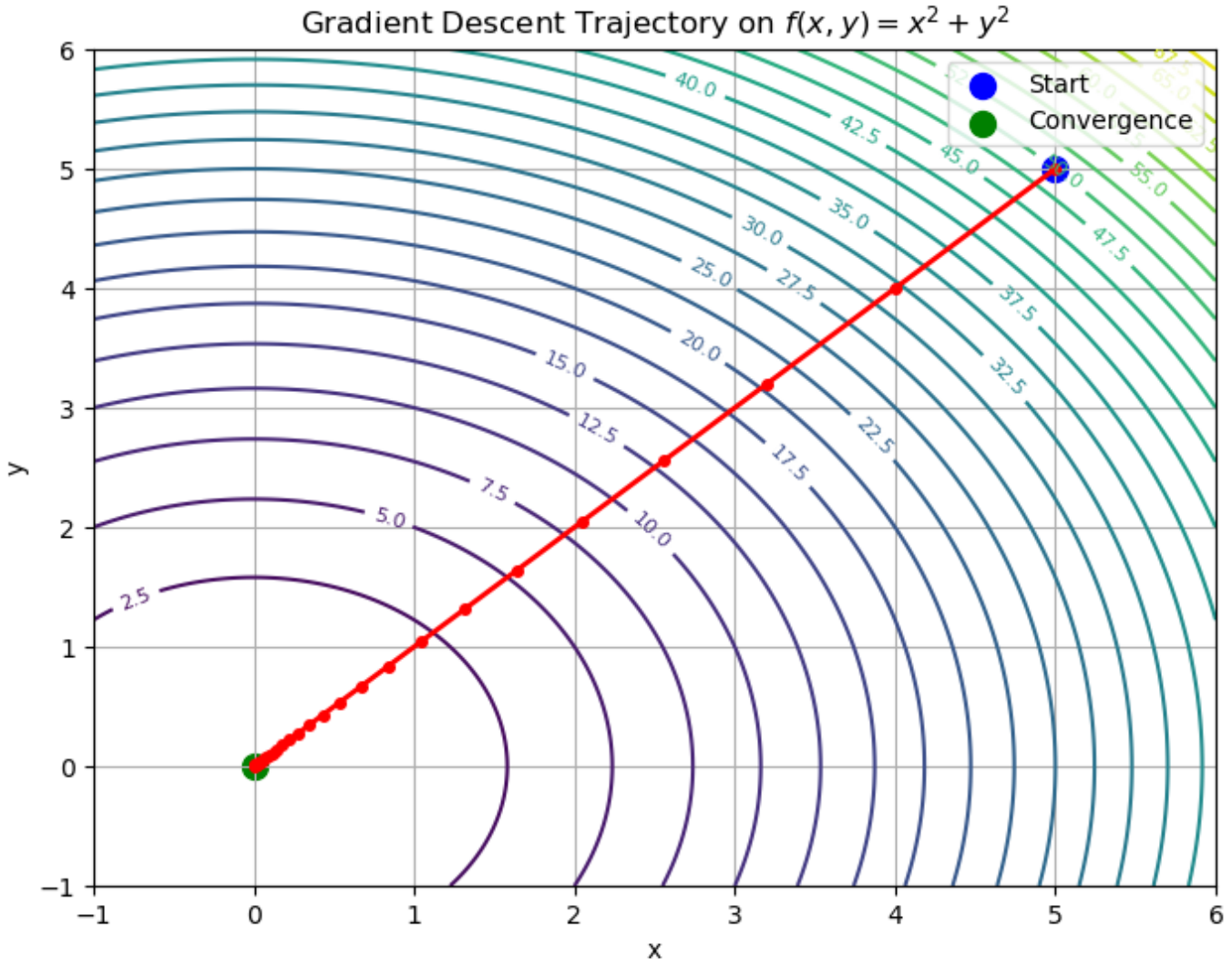
  $(\boldsymbol{x}_0 = [5.0, 5.0], \eta = 0.1)$



Gradient Descent

- $f(\boldsymbol{x}) = x_1^2 + x_2^2$
  - $\boldsymbol{x} = [x_1, x_2]$

- Repeat until convergence:
  - $\boldsymbol{x}_{t+1} = \boldsymbol{x}_t - \eta \nabla f(\boldsymbol{x}_t)$
- Where,
  - x_0: initial condition
  - η: learning rate or step size



Gradient Descent in Machine Learning

- Predicted label $\hat{y} = g(\boldsymbol{x}, \boldsymbol{w})$
- Training loss

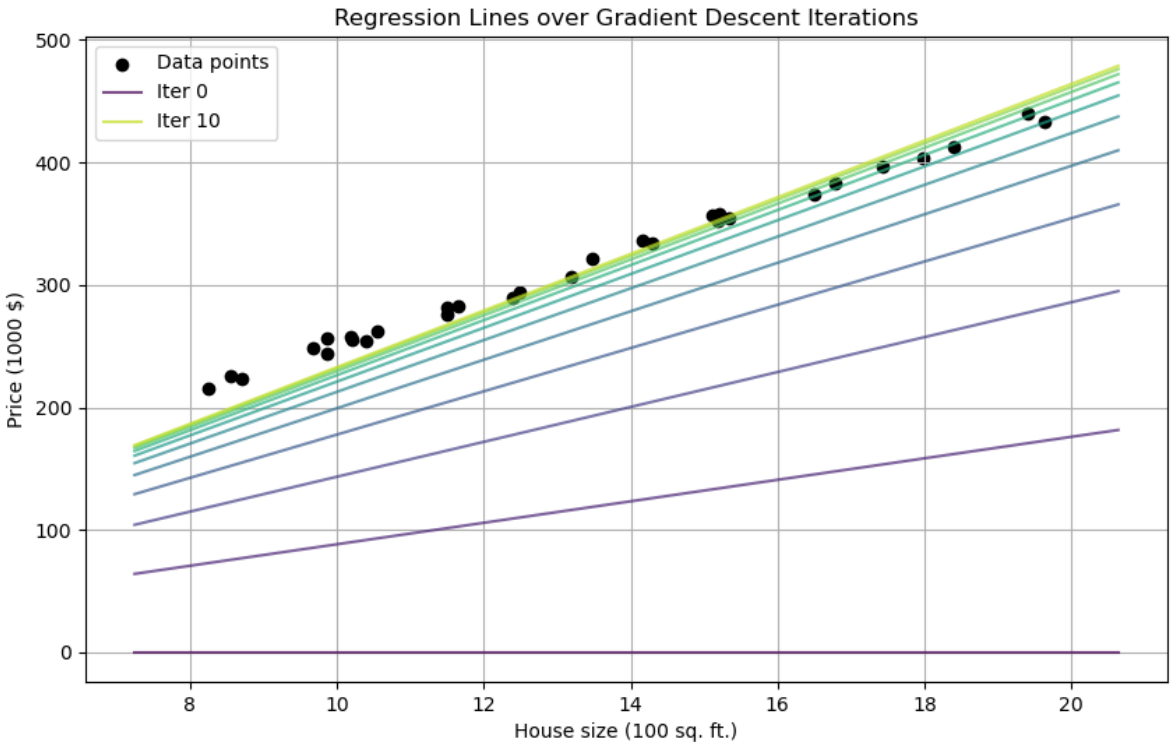
$$\mathcal{L}(\boldsymbol{w}) = \frac{1}{N_{train}} \sum_{i=1}^{N_{train}} (y_i - g(\boldsymbol{x}_i, \boldsymbol{w}))^2$$

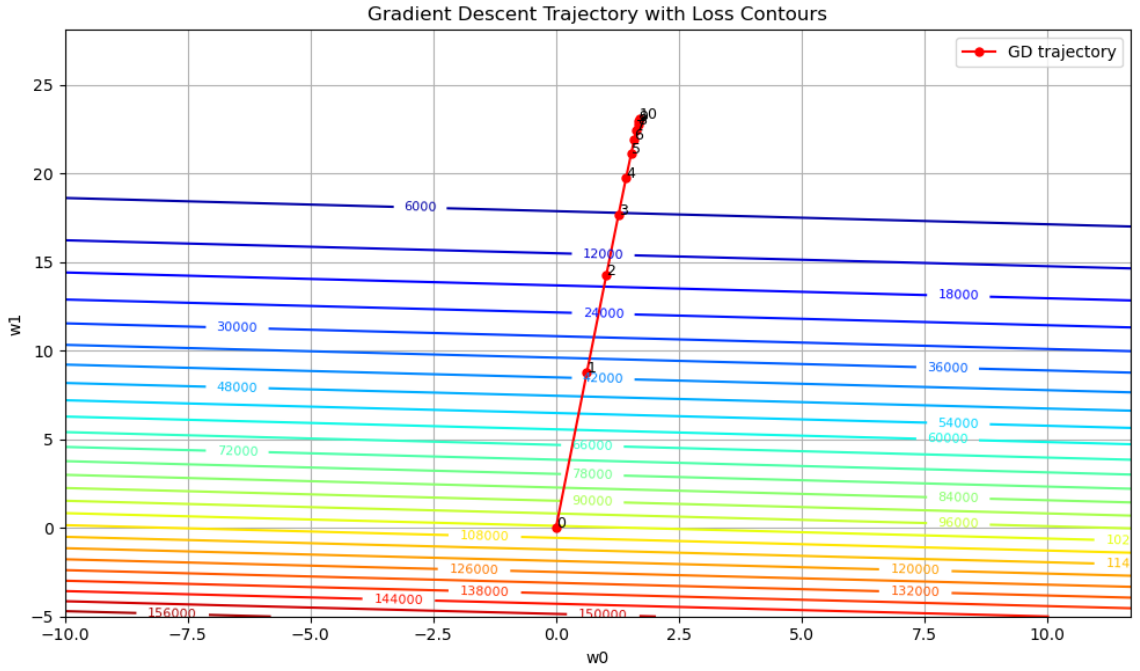
- Problem: find optimal weights that minimize $\mathcal{L}(\boldsymbol{w})$.

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$$

Example: linear regression

- $\hat{y} = g(\boldsymbol{x}, \boldsymbol{w}) = w_0 + w_1 x$





Batch Gradient Descent

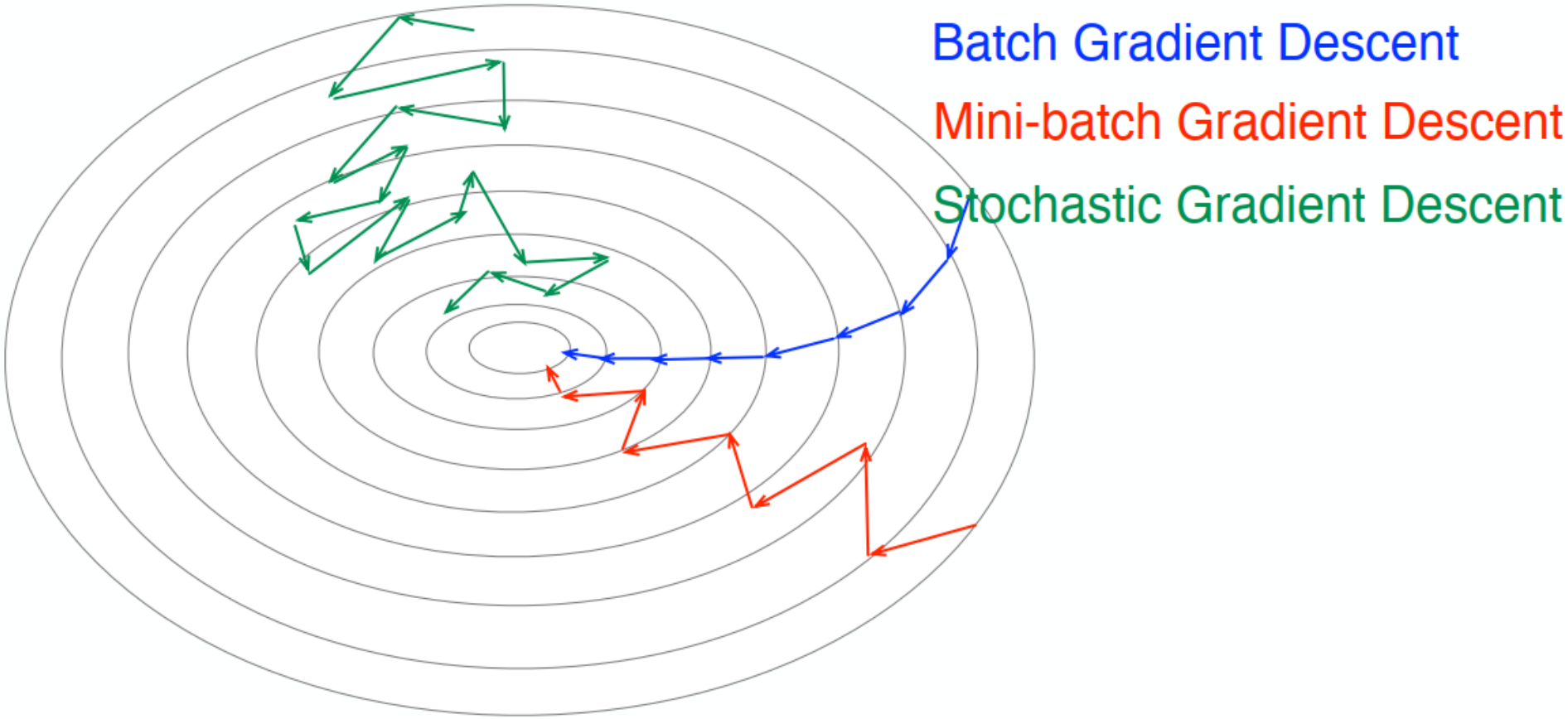
- Training loss

$$\mathcal{L}(\boldsymbol{w}) = \frac{1}{N_{train}} \sum_{i=1}^{N_{train}} (y_i - g(\boldsymbol{x}_i, \boldsymbol{w}))^2$$

- GD: $w_{t+1} = w_t - \eta \nabla \mathcal{L}(w_t)$
- Requires computing $\mathcal{L}(w_t)$ over the entire $\mathcal{D}_{train}$ in every iteration.

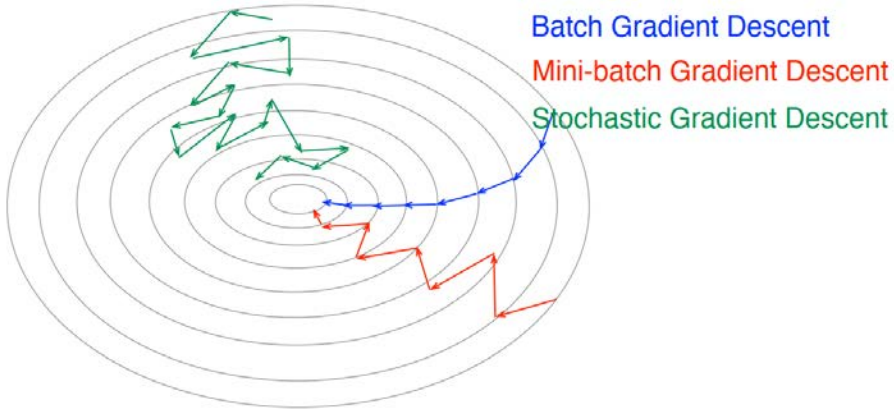
Batch Gradient Descent

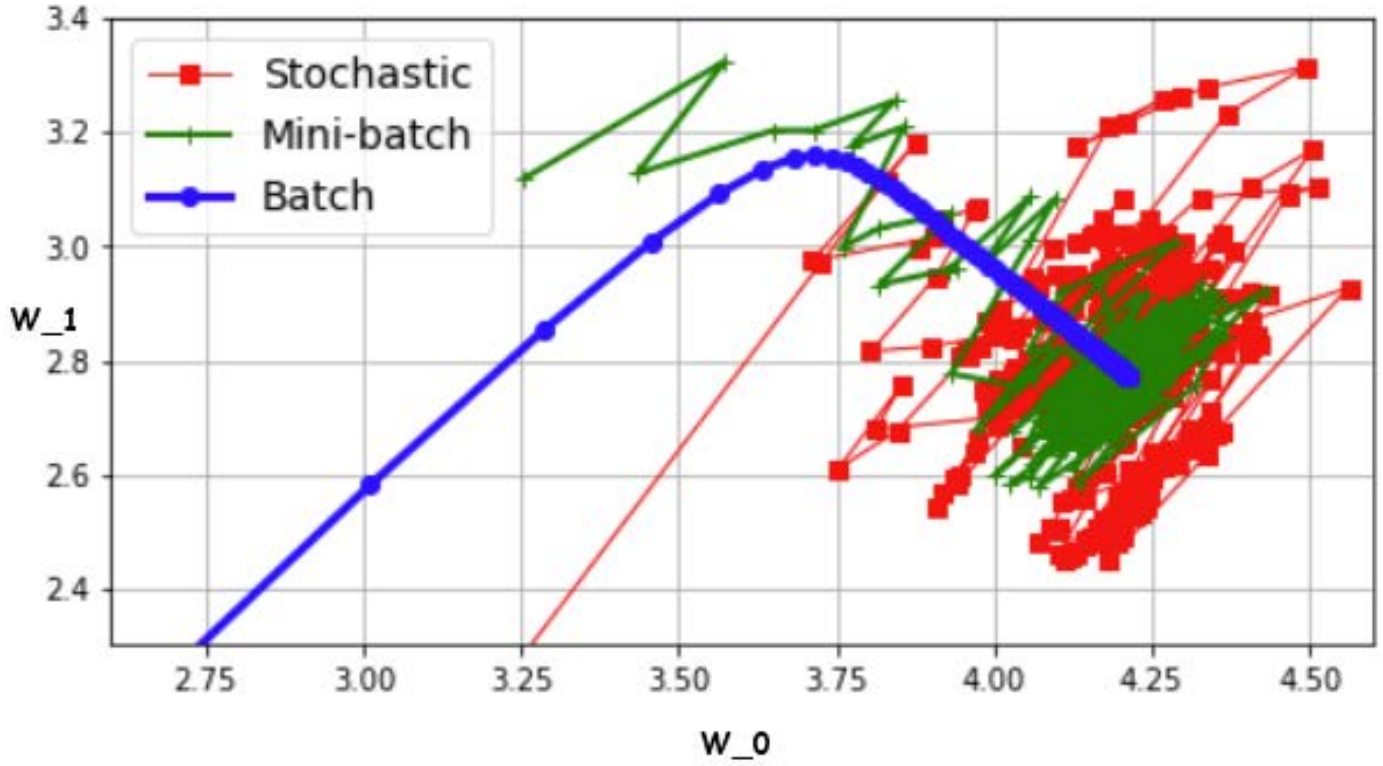
- Batch, mini-batch, and stochastic gradient descent.



Batch Gradient Descent

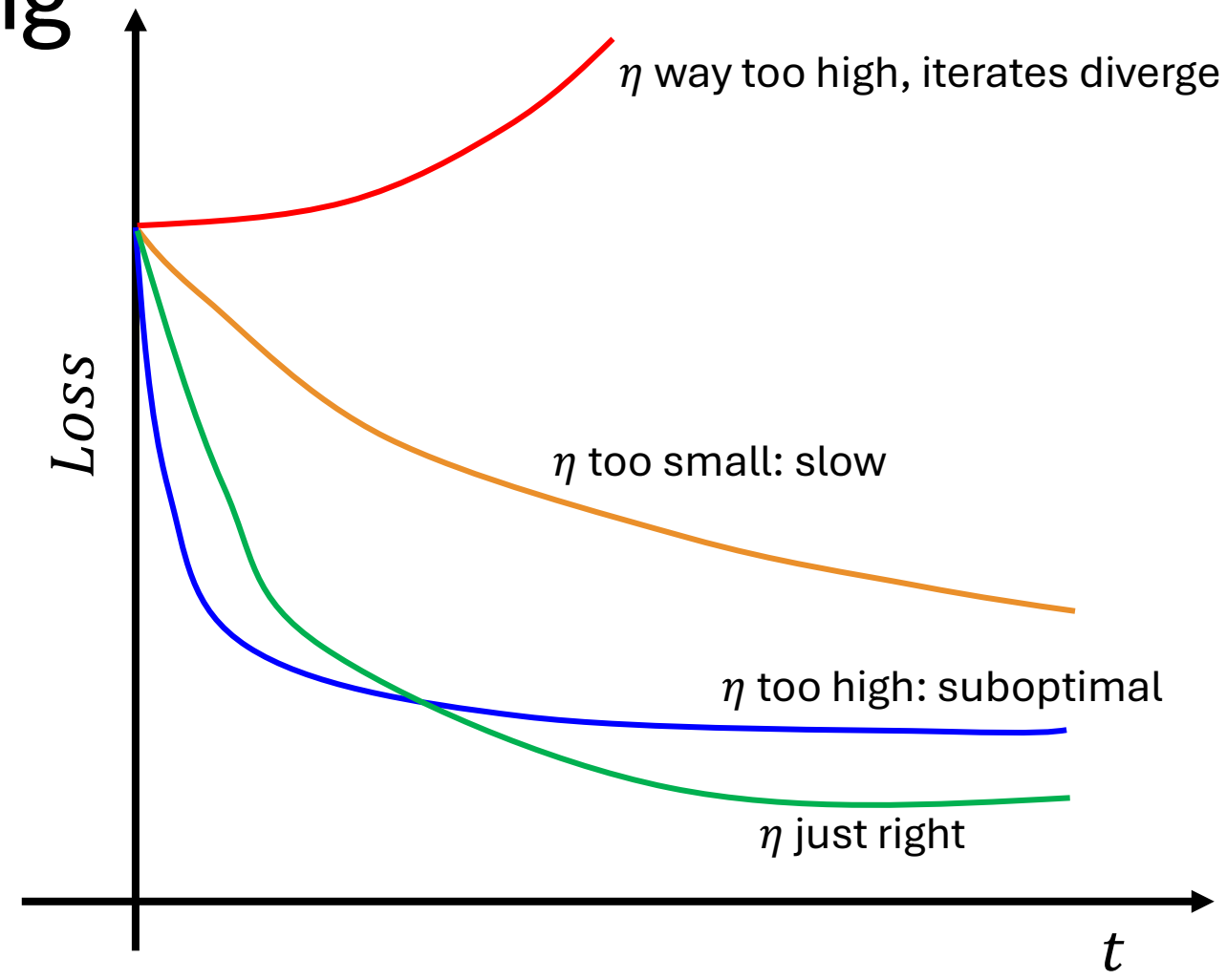
- Batch, mini-batch, and stochastic gradient descent.





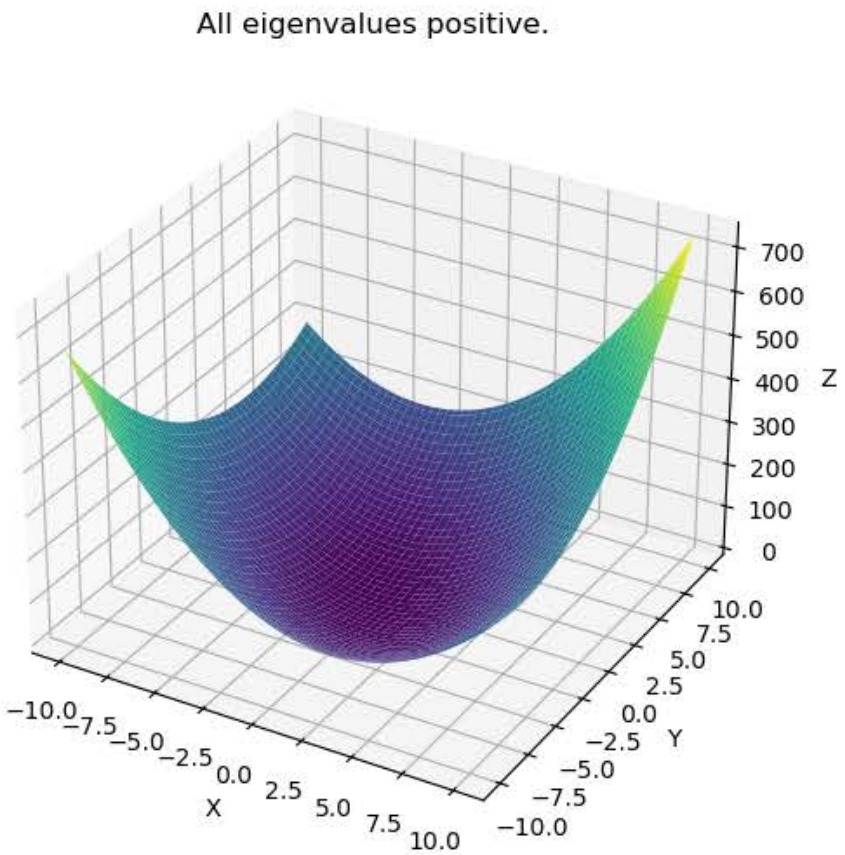
Learning rate scheduling

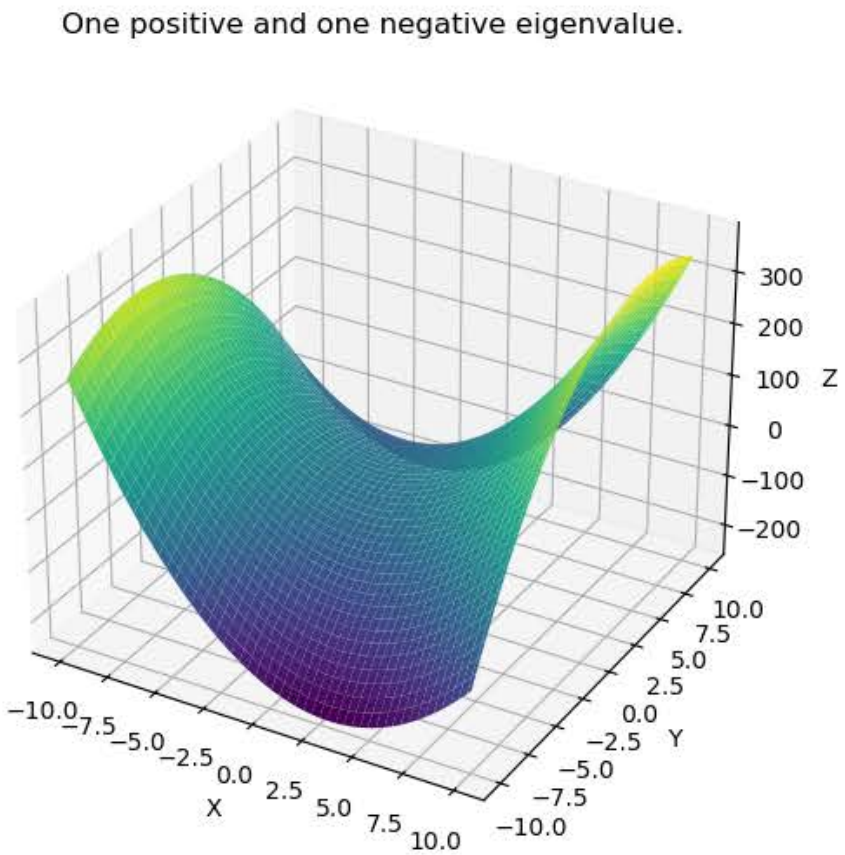
- Power scheduling
- Exponential scheduling
- Piecewise constant scheduling
- Performance scheduling



Matrix Polynomials

- Effect of eigenvalues

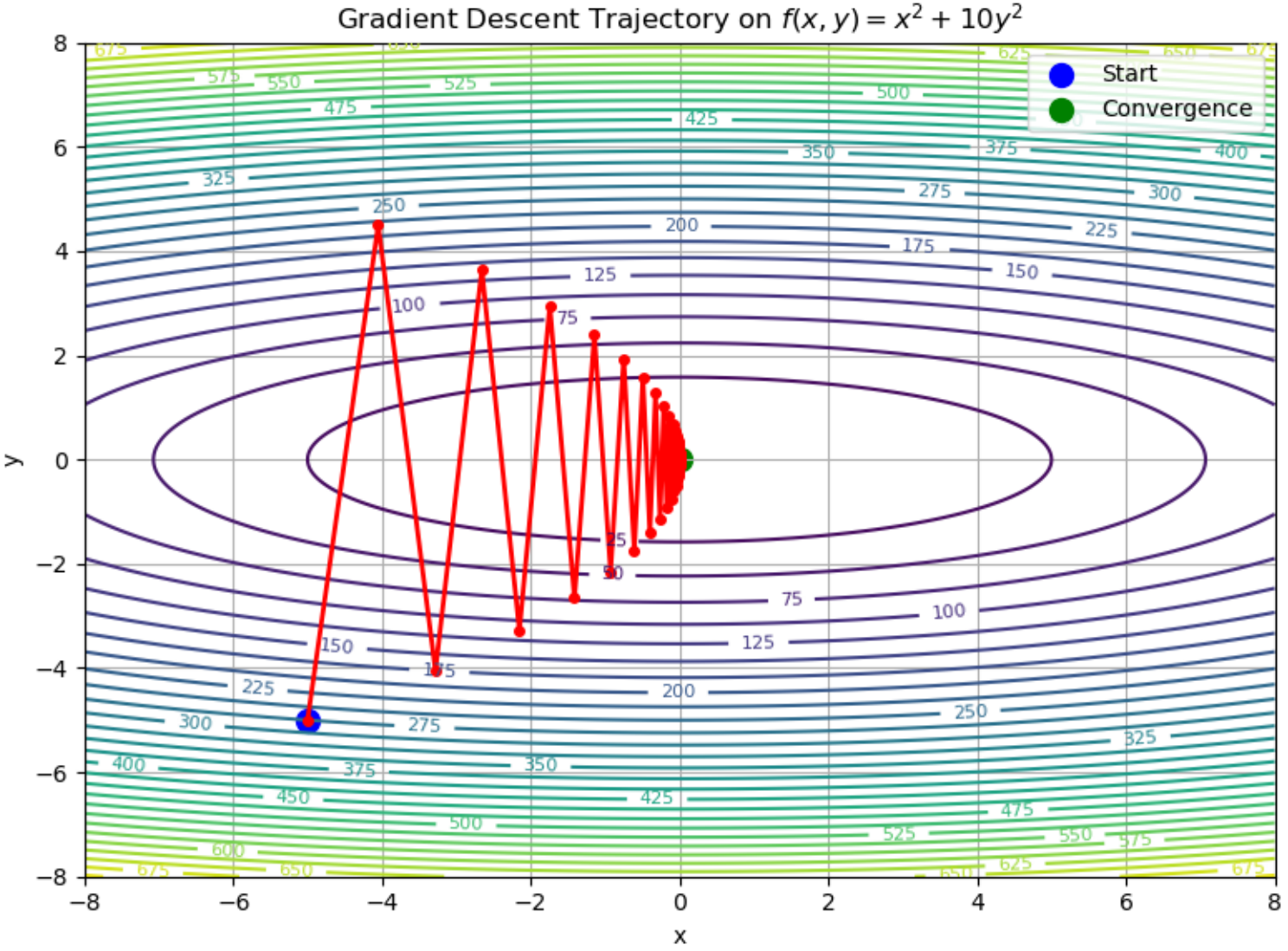




Gradient Descent

- $f(\boldsymbol{x}) = x_1^2 + 10x_2^2$
  - $\boldsymbol{x} = [x_1, x_2]$
- Repeat until convergence:
  - $\boldsymbol{x}_{t+1} = \boldsymbol{x}_t - \eta \nabla f(\boldsymbol{x}_t)$
- Where,
  - x_0: initial condition
  - η: learning rate or step size

  ($x_0 = [-5, -5]$ and $\eta = 0.095$)



Today's Class

- Differentiation and optimizing functions of 1 variable
- Iterative minimization in 1D

- Understanding the Gradient
- Gradient descent
- Issues with gradient descent

- Faster algorithms

Fast Optimizers

- GD with "momentum"
  - Two parameters: β, η
  - Boris Polyak (1964)

Repeat until convergence:

$$m_{t+1} = \beta m_t - \eta \nabla \mathcal{L}(w_t)$$
$$w_{t+1} = w_t + m$$

- m_t represents momentum

- Analogy: ball rolling down a hill
  - Escapes plateaus much faster than GD.

Adaptive Gradients (AdaGrad)

- This update is *coordinatewise*!

$$\boldsymbol{w} = [w_1, w_2, \dots, w_K]$$

- Recall that

$$\nabla \mathcal{L}(\boldsymbol{w}) = \left[{}^{\partial \mathcal{L}}/_{\partial w_1}, \dots, {}^{\partial \mathcal{L}}/_{\partial w_K} \right]$$

Repeat until convergence:

$$s_{t+1} = s_t + \left(\frac{\partial \mathcal{L}}{\partial w_{i,t}} \right)^2$$

$$w_{i,t+1} = w_{i,t} - \eta \frac{1}{\sqrt{s_t + \epsilon}} \frac{\partial \mathcal{L}}{\partial w_{i,t}}$$

RMSProp

- Accumulates only the past few gradients

Repeat until convergence:

$$s_{t+1} = \rho s_t + (1-\rho)\left(\frac{\partial \mathcal{L}}{\partial w_{i,t}}\right)^2$$

$$w_{i,t+1} = w_{i,t} - \eta \frac{1}{\sqrt{s_t + \epsilon}} \frac{\partial \mathcal{L}}{\partial w_{i,t}}$$

- $\rho = 0.9$ generally works well in practice

Adaptive Moment Estimation (Adam)

- Proposed by Diedrick Kingma and Jimmy Ba (2014)

- Combines the advantages of momentum and RMSProp.
  - Operates by taking large steps initially, and smaller steps as the function slope nears zero.

- Like momentum: keeps track of past gradients (exponentially weighted)
- Like RMSProp: past squared partial derivatives (exponentially weighted)